

并查集

1. 将两个集合合并
2. 询问两个元素是否在同一集合中

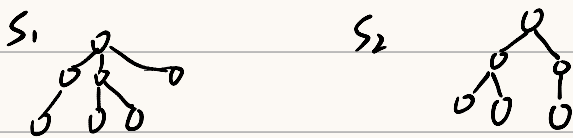
暴力: 用 $belong[]$ 数组存每个元素属于哪个集合

对于 2 操作 $belong[a] == belong[b]$ $O(1)$

但 1 操作, 就要把所有 $belong[x] = k$ 的都修改为另一个集合

并查集可以在近乎 $O(1)$ 时间完成 1、2.

基本原理: 用树的形式维护每个集合



每个集合的编号为其根结点编号

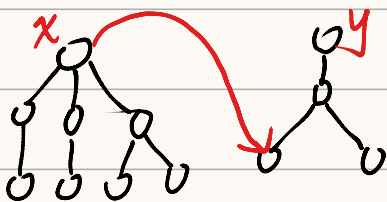
对于每个点, 都有其父结点的编号, $p[x]$ 为其父结点编号

1° 对于非 root 的结点, $p[x] \neq x$, 我们会令 $p[root] = root$

2° 求 x 集合编号: $while (p[x] \neq x) \ x = p[x]$

不断向上走, 直到走到该树的树根

3° 如何合并两集合? 把一个树的树根插到另一棵树上



$p[x] = y$

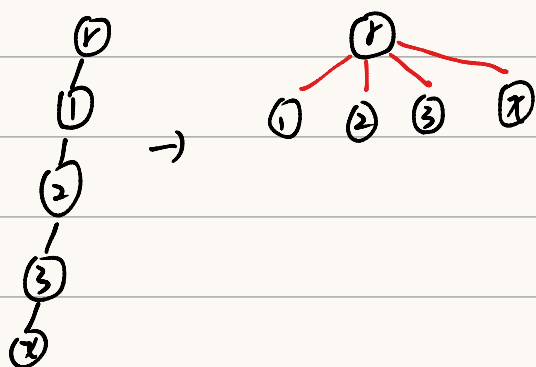
但这样, 随着不断合并, 树高 ↑

求 x 所在集合所需跳转次数 ↑

优化: 在 x 向上寻找的过程中, 把路径上所有结点直接与根
结点连接

路径压缩

这样时间复杂度基本可以
视为 $O(1)$ 了。



代码实现:

1° 查找所在集合

`int find(x)` 返回 x 的祖宗结点 + 路径压缩

`if (p[x] != x) p[x] = find(p[x])`

`return p[x]` 递归, 利用函数定义

父结点 = 祖宗结点, 再返回父结点

2° 合并

`void merge(x, y)`

`a = find(x) b = find(y)`

`if (a != b) p[a] = b`

836. 合并集合

🏠 题目

📋 提交记录

💬 讨论

📖 题解

🎥 视频讲解

一共有 n 个数，编号是 $1 \sim n$ ，最开始每个数各自在一个集合中。

现在要进行 m 个操作，操作共有两种：

- `M a b`，将编号为 a 和 b 的两个数所在的集合合并，如果两个数已经在同一个集合中，则忽略这个操作；
- `Q a b`，询问编号为 a 和 b 的两个数是否在同一个集合中；

输入格式

第一行输入整数 n 和 m 。

接下来 m 行，每行包含一个操作指令，指令为 `M a b` 或 `Q a b` 中的一种。

输出格式

对于每个询问指令 `Q a b`，都要输出一个结果，如果 a 和 b 在同一集合内，则输出 `Yes`，否则输出 `No`。

每个结果占一行。

数据范围

$$1 \leq n, m \leq 10^5$$

输入样例：

```
4 5
M 1 2
M 3 4
Q 1 2
Q 1 3
Q 3 4
```

输出样例：

```
Yes
No
Yes
```

难度：

简单

时/空限制：1s / 64MB

总通过数：128562

总尝试数：209896

来源：

模板题

AcWing

算法标签

```
1 #include <stdio.h>
2 #include <string.h>
3
4 const int N = 1e5 + 10;
5 int n, m;
6 int P[N]; // parent数组，标记每个数所在集合
7
8 int find(int x) {
9     // 函数定义：找到x的祖宗结点并进行路径压缩
10    if(P[x] != x) P[x] = find(P[x]);
11    return P[x];
12    //如果x父节点不是root，那么利用函数定义，将父节点
13    // 替换为root，并返回父节点
14 }
15
16 void merge(int x, int y) {
17     int a = find(x), b = find(y);
18     if(a != b) P[a] = b;
19     return ;
20 }
21
22
23 int main() {
24     scanf("%d%d", &n, &m);
25     for(int i = 0 ; i < n; i++) P[i] = i;
26     // 初始化这一步不能忘，最开始每个元素都是一个单独的集合
27     char choose[5];
28     int x, y;
29     for(int i = 0; i < m; i ++){
30         scanf("%s", choose);
31         // 这里虽然是单个字符，但是我们还是选择读字符串，
32         // scanf读字符的时候会读空格和换行符，读字符串会跳过，省事且不容易出错
33         if(strcmp(choose, "M") == 0) {
34             scanf("%d%d", &x, &y);
35             merge(x, y);
36         } else {
37             scanf("%d%d", &x, &y);
38             if(find(x) == find(y)) printf("Yes\n");
39             else printf("No\n");
40         }
41     }
42
43     return 0;
44 }
```

837. 连通块中点的数量

题目 提交记录 讨论 题解 视频讲解

给定一个包含 n 个点（编号为 $1 \sim n$ ）的无向图，初始时图中没有边。

现在要进行 m 个操作，操作共有三种：

1. **C a b**，在点 a 和点 b 之间连一条边， a 和 b 可能相等；
2. **Q1 a b**，询问点 a 和点 b 是否在同一个连通块中， a 和 b 可能相等；
3. **Q2 a**，询问点 a 所在连通块中点的数量；

输入格式

第一行输入整数 n 和 m 。

接下来 m 行，每行包含一个操作指令，指令为 **C a b**，**Q1 a b** 或 **Q2 a** 中的一种。

输出格式

对于每个询问指令 **Q1 a b**，如果 a 和 b 在同一个连通块中，则输出 **Yes**，否则输出 **No**。

对于每个询问指令 **Q2 a**，输出一个整数表示点 a 所在连通块中点的数量

每个结果占一行。

数据范围

$1 \leq n, m \leq 10^5$

输入样例：

```
5 5
C 1 2
Q1 1 2
Q2 1
C 2 5
Q2 5
```

输出样例：

```
Yes
2
3
```

难度：

简单

时空限制：

1s / 64MB

总通过数：

90914

总尝试数：

181161

来源：

模板题

算法标签

```
1 #include <stdio.h>
2 #include <string.h>
3
4 const int N = 1e5 + 10;
5 int n, m;
6 int P[N], cnt[N]; // 记录祖宗节点和所在集合点的数量
7
8 int find(int x) {
9     if (P[x] != x) P[x] = find(P[x]);
10    return P[x];
11 }
12
13 void merge(int x, int y) {
14     int a = find(x), b = find(y);
15     if (a != b) {
16         P[a] = b;
17         cnt[b] += cnt[a];
18         // a b不在一个集合的时候才更改cnt
19     }
20     return ;
21 }
22
23
24 int main() {
25     scanf("%d%d", &n, &m);
26     // 初始化
27     for (int i = 0; i < n; i++) {
28         P[i] = i;
29         cnt[i] = 1;
30     }
31
32     char choose[5];
33     int x, y;
34
35     for (int i = 0; i < m; i++) {
36         scanf("%s", choose);
37         if (strcmp(choose, "C") == 0) {
38             scanf("%d%d", &x, &y);
39             merge(x, y);
40         } else if (strcmp(choose, "Q1") == 0) {
41             scanf("%d%d", &x, &y);
42             if (find(x) == find(y)) printf("Yes\n");
43             else printf("No\n");
44         } else {
45             scanf("%d", &x);
46             printf("%d\n", cnt[find(x)]);
47         }
48     }
49
50     return 0;
51 }
52
```

240. 食物链

🏠 题目

📋 提交记录

💬 讨论

📖 题解

🎥 视频讲解

动物王国中有三类动物 A, B, C ，这三类动物的食物链构成了有趣的环形。
 A 吃 B , B 吃 C , C 吃 A 。

现有 N 个动物，以 $1 \sim N$ 编号。

每个动物都是 A, B, C 中的一种，但是我们并不知道它到底是哪一种。

有人用两种说法对这 N 个动物所构成的食物链关系进行描述：

第一种说法是 $1\ X\ Y$ ，表示 X 和 Y 是同类。

第二种说法是 $2\ X\ Y$ ，表示 X 吃 Y 。

此人对 N 个动物，用上述两种说法，一句接一句地说出 K 句话，这 K 句话有的是真的，有的是假的。

当一句话满足下列三条之一时，这句话就是假话，否则就是真话。

- 当前的话与前面的某些真的话冲突，就是假话；
- 当前的话中 X 或 Y 比 N 大，就是假话；
- 当前的话表示 X 吃 X ，就是假话。

你的任务是根据给定的 N 和 K 句话，输出假话的总数。

输入格式

第一行是两个整数 N 和 K ，以一个空格分隔。

以下 K 行每行是三个正整数 D, X, Y ，两数之间用一个空格隔开，其中 D 表示说法的种类。

若 $D = 1$ ，则表示 X 和 Y 是同类。

若 $D = 2$ ，则表示 X 吃 Y 。

输出格式

只有一个整数，表示假话的数目。

数据范围

$1 \leq N \leq 50000$,
 $0 \leq K \leq 100000$

输入样例：

```
100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

输出样例：

```
3
```

难度：中等

时/空限制：1s / 64MB

总通过数：72887

总尝试数：160006

来源：
《算法竞赛进阶指南》模板题
NOI2001POJ1182
kuangbin专题
《信息学奥赛一本通》语言及算法基础篇

算法标签 ▼

```

1 #include <stdio.h>
2 #include <math.h>
3
4 const int N = 5e4 + 10;
5 int n, k;
6 int res = 0;
7 int P[N], cnt[N]; // 记录父节点和到父结点的距离
8 /*
9 注意! cnt[x] 表示的是x到父节点的距离, 但是find x之后路径压缩
10 x的父结点就是对应集合的根节点!
11
12 cnt[x] % 3 的结果 x 和根节点的关系
13 0 x 与根节点同类
14 1 x 吃根节点
15 2 根节点吃 x
16 */
17 // (cnt[x] - cnt[y]) % 3 == 0 x y同类
18 // (cnt[x] - cnt[y] - 1) % 3 == 0 x吃y
19 // 思路: 我们把能确定关系的结点放到同一个集合里面
20
21
22 int find(int x) {
23     // 函数定义: 返回x的祖宗结点, 并路径压缩
24     // 压缩完之后, P[x]就是当前集合的root
25     // 所以cnt[x] = cnt[x] + cnt[P[x]]
26     // 逻辑上x到根节点的距离应该是 x到原父结点距离 + 原父结点到根结点距离
27     if (P[x] != x) {
28         // 第一步: 先递归找到P[x]的根节点t, 同时完成P[x]的路径压缩和cnt[P[x]]的更新
29         int t = find(P[x]);
30         // 第二步: 此时P[x]已经指向根节点t, cnt[P[x]]是「P[x]到根节点t的最终距离」
31         // 所以cnt[x] = x到原父节点P[x]的距离 + 原父节点P[x]到根节点t的距离
32         cnt[x] += cnt[P[x]];
33         // 第三步: 把x的父节点直接指向根节点t, 完成x的路径压缩
34         P[x] = t;
35     }
36     return P[x];
37 }
38
39 /*
40 当x和y关系未确定时, 我们把x集合连接到y集合上
41 逻辑上来讲, 当连接完成之后, x到根节点的距离就是 x到xx + xx到yy, 即 cnt[x] + cnt[xx];
42 场景 1 (a=1, x 和 y 同类): 要求 (cnt[x] + cnt[xx]) % 3 = cnt[y] % 3 → 解得cnt[xx] = cnt[y] - cnt[x];
43 场景 2 (a=2, x 吃 y): 要求 (cnt[x] + cnt[xx]) % 3 = (cnt[y] + 1) % 3 → 解得cnt[xx] = cnt[y] + 1 - cnt[x];
44
45 */

```

```

47 int main() {
48     scanf("%d%d", &n, &k);
49
50     for (int i = 1; i <= n; i++) {
51         P[i] = i; cnt[i] = 0;
52     } // 编号从1开始的
53
54     int a, x, y;
55     for (int i = 0; i < k; i++) {
56         scanf("%d%d%d", &a, &x, &y);
57         if (x > n || y > n) {
58             res++;
59             continue;
60         }
61
62         int xx = find(x), yy = find(y);
63         if (a == 1) {
64             if (xx != yy) { // x 和 y 的关系还未确定, 在a = 1时, 说明是同类
65                 P[xx] = yy;
66                 cnt[xx] = cnt[y] - cnt[x]; // 如果cnt[xx]因为累计太多爆了
67                 // 可以把每个cnt都约束在0 1 2三个数
68                 // cnt[xx] = ((cnt[y] - cnt[x]) % 3 + 3) % 3;
69             } else { // x y关系已经确定, 现在就判断是否是同类
70                 if ((cnt[x] - cnt[y]) % 3 != 0) res++;
71             }
72         } else {
73             if (xx != yy) { // x 和 y 的关系还未确定, 在a = 2时, 说明是x吃y
74                 P[xx] = yy;
75                 cnt[xx] = cnt[y] + 1 - cnt[x];
76                 // cnt[xx] = ((cnt[y] + 1 - cnt[x]) % 3 + 3) % 3;
77             } else {
78                 if ((cnt[x] - cnt[y] - 1) % 3 != 0) res++;
79             }
80         }
81     }
82     printf("%d\n", res);
83
84     return 0;
85 }

```

